and the most common bugs I've found in production PHP code occur when multi-byte encoded text is passed through core string functions (which are not aware of encoding at all). To that end, this section will also present `mbstring` equivalent functions to core functions (where they exist), with comparisons between the core and `mbstring` forms and examples of how the core functions tend to fail when they're tasked with processing multi-byte data. One of my goals in life is to never again have to fix code that should have used, say, `mb_strlen()` instead of `strlen()`. It's important to have dreams.

Anyway, on with the show…

Comparison and Analysis

The simplest way to compare two strings for equivalence is, of course, to use the comparison operators == or ===. This will trigger a byte-by-byte comparison and return `true` if the two buffers are identical (the weakly-typed == will of course coerce data types when comparing a string to a non-string). This is useful for comparing strings that you might use, say, as internal values in a program, but not so useful when you introduce natural language or user input (with associated variance in typing ability or degrees of hostility and malicious intent). Furthermore, it might well be the case that it is only a portion of the string that is of interest, or the frequency with which certain characters or sub-strings occur.

Writing your own functions to iterate a (string) buffer looking for patterns of bytes is perfectly doable in PHP. To iterate over the bytes in a string buffer, you can simply treat the string as an array:

```
$str = 'PHP is a glorious language!';

$count = strlen($str);
for ($i = 0; $i < $count; $i++) {
    print $str[$i];
}

print PHP_EOL;

Output:
PHP is a glorious language!
```